

Notas Práticas e Exercícios 3

Disciplina: Programação Orientada a Objetos

{*Anotações para uso pessoal}

1 Classes Aninhadas em Java

Classes aninhadas em Java referem-se a uma classe definida dentro de outra classe. Existem quatro tipos de classes aninhadas em Java: classes aninhadas internas, classes aninhadas estáticas, classes locais e classes anônimas. Cada tipo tem suas próprias características e propósitos.

1.1 Classes Aninhadas Internas

As classes aninhadas internas, também conhecidas como classes internas não estáticas, são definidas dentro de uma classe externa e têm acesso aos membros da classe externa, incluindo membros privados. Para instanciar uma classe interna, é necessário ter uma instância da classe externa.

A sintaxe para definir uma classe interna é a seguinte:

```
1 class ClasseExterna {  
2     // membros da classe externa  
3  
4     class ClasseInterna {  
5         // membros da classe interna  
6     }  
7 }
```

1.2 Classes Aninhadas Estáticas

As classes aninhadas estáticas, também conhecidas como classes estáticas internas, são semelhantes às classes internas, mas são definidas como estáticas. Elas não precisam de uma instância da classe externa para serem instanciadas e têm acesso apenas aos membros estáticos da classe externa.

A sintaxe para definir uma classe aninhada estática é a seguinte:

```
1 class ClasseExterna {  
2     // membros da classe externa  
3  
4     static class ClasseInterna {  
5         // membros da classe interna  
6     }  
7 }
```

1.3 Classes Locais

As classes locais são definidas dentro de um método ou bloco de código e têm acesso aos membros da classe externa e aos membros finais ou efetivamente finais declarados no escopo em que foram definidas. As classes locais são usadas principalmente para dividir um método em partes menores e mais fáceis de gerenciar.

A sintaxe para definir uma classe local é a seguinte:

```
1 class ClasseExterna {
2     // membros da classe externa
3
4     public void metodoExemplo() {
5
6         class ClasseLocal {
7             // membros da classe
8         }
9
10        ClasseLocal minhaClasseLocal = new ClasseLocal();
11        // usar a classe local aqui
12
13    }
14
15 }
```

1.4 Classes Anônimas

As classes anônimas são definidas como expressões e não como declarações de classe. Elas são usadas principalmente para implementar interfaces e classes abstratas em uma única linha de código. As classes anônimas não têm nome e não podem ser reutilizadas em outras partes do código.

A sintaxe para definir uma classe anônima é a seguinte:

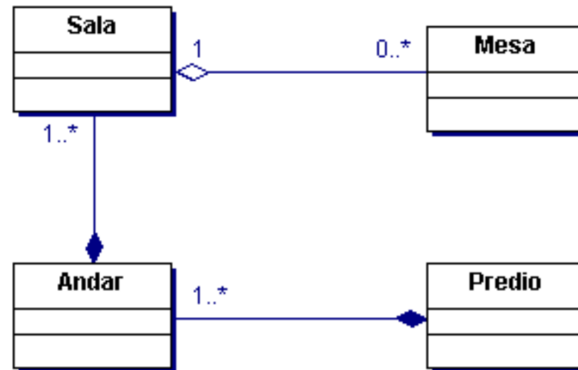
```
1 InterfaceExemplo minhaInterface = new InterfaceExemplo() {
2     // membros da classe anônima
3 };
```

2 Exemplo UML para Java

Considerando o diagrama da Figura 1. No diagrama UML, Predio compõe Andar com multiplicidade 1..*, Andar compõe Sala com multiplicidade 1..*, e Sala agrega Mesa com multiplicidade 0..*, indicando que um prédio contém um ou mais andares, cada andar contém uma ou mais salas, e cada sala pode conter zero ou mais mesas. A implementação em Java deste diagrama pode ser realizada como abaixo.

```
1 // Classe pública Mesa representa uma mesa que pode ser parte de uma sala.
2 public class Mesa {
3     // Campo privado que mantém uma referência para a Sala à qual esta mesa
4     // pode pertencer.
5     // A referência pode ser nula se a mesa ainda não estiver associada a
6     // nenhuma sala.
7     private Sala sala;
8
9     // Construtor padrão sem argumentos. Cria uma mesa sem estar associada
10    // a uma sala.
```

Figura 1: Diagrama UML de classes.



```
8 public Mesa(){
9     // Este construtor não inicializa a sala, então 'sala' permanece
    nulo até que 'setSala' seja chamado.
10 }
11
12 // Construtor sobrecarregado que aceita uma Sala como argumento.
13 // Isto associa diretamente a nova Mesa à Sala fornecida.
14 public Mesa(Sala sala) {
15     this.sala = sala; // Armazena a referência da Sala fornecida.
16 }
17
18 // Método para definir ou mudar a Sala associada a esta Mesa.
19 public void setSala(Sala sala){
20     this.sala = sala; // Atualiza a referência da Sala para a fornecida
    como argumento.
21 }
22
23 // outros métodos...
24 // Outros métodos podem ser adicionados aqui para expandir a
    funcionalidade da classe Mesa.
25 }
26
27 // Classe pública Sala que representa uma sala dentro de um andar de um
    edifício.
28 public class Sala {
29
30     // Uma lista privada que contém objetos do tipo Mesa.
31     // Esta lista mantém o registro de todas as mesas dentro desta sala.
32     private List<Mesa> mesas = new ArrayList<>();
33
34     // Referência privada para o objeto Andar ao qual esta Sala pertence.
35     private Andar andar;
36
37     // Construtor da classe Sala. Cada sala precisa ter uma referência ao
    andar a que pertence.
38     public Sala(Andar andar) {
```

```

39         this.andar = andar; // Armazena a referência do andar fornecido no
           construtor.
40     }
41
42     // Método para adicionar uma mesa à sala.
43     // Antes de adicionar a mesa à lista, ele configura a sala desta mesa
           para esta sala.
44     public void addMesa(Mesa mesa) {
45         mesa.setSala(this); // Define a sala atual como a sala da mesa.
46         mesas.add(mesa); // Adiciona a mesa à lista de mesas desta sala.
47     }
48
49     // Método para obter uma mesa específica pelo seu índice (numeroMesa)
           na lista de mesas.
50     // Se o índice fornecido for válido, ele retorna a mesa correspondente.
51     public Mesa getMesa(int numeroMesa) {
52         return mesas.get(numeroMesa); // Retorna a mesa com base no índice
           fornecido.
53     }
54
55     // Método que retorna o número total de mesas na sala.
56     // Isso é feito retornando o tamanho da lista de mesas.
57     public int getNumeroDeMesas() {
58         return mesas.size(); // Retorna a quantidade de mesas na lista.
59     }
60
61     // outros métodos...
62     // Aqui, outros métodos relacionados à classe Sala podem ser definidos.
63 }
64
65 //Classe Publica Andar
66 public class Andar {
67
68     // Lista privada que contém as salas associadas a este andar.
69     private List<Sala> salas = new ArrayList<>();
70
71     // Referência ao prédio ao qual este andar pertence.
72     private Predio predio;
73
74     // Construtor da classe Andar. Quando um novo andar é criado, ele
           automaticamente adiciona uma sala.
75     public Andar(Predio predio) {
76         this.predio = predio; // Associa o andar ao prédio fornecido como
           argumento.
77         addSala(); // Adiciona uma sala ao andar durante a criação do andar.
78     }
79
80     // Método para adicionar uma nova sala a este andar.
81     public void addSala() {
82         salas.add(new Sala(this)); // Cria uma nova sala e a adiciona à
           lista de salas.

```

```

83     }
84
85     // Método para adicionar uma mesa a uma sala específica neste andar.
86     public void addMesa(int numeroSala, Mesa mesa) {
87         if (numeroSala >= 0 && numeroSala < salas.size()) { // Verifica se
88             o número da sala é válido.
89             Sala sala = salas.get(numeroSala); // Obtém a sala pelo seu
90             número.
91             sala.addMesa(mesa); // Adiciona a mesa à sala.
92         } else {
93             throw new IndexOutOfBoundsException("Número do andar
94             inválido."); // Lança uma exceção se o número da sala for
95             inválido.
96         }
97     }
98
99     // Método para obter o número total de salas neste andar.
100    public int getNumeroSalas() {
101        return salas.size(); // Retorna o tamanho da lista de salas, ou
102        seja, o número de salas.
103    }
104
105    // Método para obter o número de mesas em uma sala específica neste
106    andar.
107    public int getNumeroMesasNaSala(int numeroDaSala) {
108        if (numeroDaSala >= 0 && numeroDaSala < salas.size()) { // Verifica
109            se o número da sala é válido.
110            Sala sala = salas.get(numeroDaSala); // Obtém a sala pelo seu
111            número.
112            return sala.getNumeroDeMesas(); // Retorna o número de mesas na
113            sala.
114        } else {
115            throw new IndexOutOfBoundsException("Número da sala
116            inválido."); // Lança uma exceção se o número da sala for
117            inválido.
118        }
119    }
120
121    // outros métodos...
122 }
123
124 // Classe pública Predio representa um prédio com vários andares.
125 public class Predio {
126     // Lista privada de andares, controlando os andares que compõem o
127     prédio.
128     private List<Andar> andares = new ArrayList<>();
129
130     // Construtor padrão para Predio. Cria um prédio com um andar por
131     padrão.
132     public Predio() {
133         this(1); // Chama o outro construtor passando 1 como número padrão

```

```

121         de andares.
122     }
123     // Construtor sobrecarregado para Predio que aceita um número de
124     // andares a serem criados.
125     public Predio(int numeroAndares) {
126         for (int i = 0; i < numeroAndares; i++){
127             andares.add(new Andar(this)); // Cria e adiciona o número
128             // especificado de andares à lista.
129         }
130     }
131     // Método para adicionar um andar adicional ao prédio.
132     public void adicionarAndar() {
133         Andar novoAndar = new Andar(this); // Cria um novo andar associado
134         // a este prédio.
135         andares.add(novoAndar); // Adiciona o novo andar à lista de andares.
136     }
137     // Método para adicionar uma sala a um andar específico baseado em seu índice.
138     public void adicionarSalaAoAndar(int numeroDoAndar) {
139         if (numeroDoAndar >= 0 && numeroDoAndar < andares.size()) { //
140             // Verifica se o índice do andar é válido.
141             Andar andar = andares.get(numeroDoAndar); // Obtém o andar pelo
142             // seu índice.
143             andar.addSala(); // Chama o método addSala do andar para
144             // adicionar uma nova sala a ele.
145         } else {
146             System.out.println("Número do andar inválido."); // Caso o índice
147             // seja inválido, imprime uma mensagem de erro.
148         }
149     }
150     // Método para adicionar uma mesa a uma sala específica em um andar
151     // específico.
152     public void adicionarMesaSala(int numeroDoAndar, int numeroSala, Mesa
153     mesa) {
154         if (numeroDoAndar >= 0 && numeroDoAndar < andares.size()) { //
155             // Verifica se o índice do andar é válido.
156             Andar andar = andares.get(numeroDoAndar); // Obtém o andar pelo
157             // seu índice.
158             andar.addMesa(numeroSala, mesa); // Chama o método addMesa do
159             // andar para adicionar uma mesa à sala especificada.
160         } else {
161             System.out.println("Número do andar inválido."); // Caso o índice
162             // seja inválido, imprime uma mensagem de erro.
163         }
164     }
165     // Método para obter o número total de salas em um andar específico.

```

```

157     public int obterNumeroDeSalasNoAndar(int numeroDoAndar) {
158         if (numeroDoAndar >= 0 && numeroDoAndar < andares.size()) { //
159             Verifica se o índice do andar é válido.
160             return andares.get(numeroDoAndar).getNumeroSalas(); // Retorna
161                 o número de salas no andar especificado.
162         } else {
163             System.out.println("Número do andar inválido."); // Caso o í
164                 ndice seja inválido, imprime uma mensagem de erro.
165             return -1; // Retorna -1 para indicar que houve um erro.
166         }
167     }
168
169     // Método para obter o número de mesas em uma sala específica em um
170     // andar específico.
171     public int getNumeroMesasporSala(int numeroDoAndar, int numeroDaSala) {
172         if (numeroDoAndar >= 0 && numeroDoAndar < andares.size()) { //
173             Verifica se o índice do andar é válido.
174             Andar andar = andares.get(numeroDoAndar); // Obtém o andar pelo
175                 seu índice.
176             return andar.getNumeroMesasNaSala(numeroDaSala); // Retorna o
177                 número de mesas na sala especificada.
178         } else {
179             System.out.println("Número do andar inválido."); // Caso o í
180                 ndice seja inválido, imprime uma mensagem de erro.
181             return -1; // Retorna -1 para indicar que houve um erro.
182         }
183     }
184
185     // outros métodos...
186     // Aqui você pode adicionar mais métodos relacionados à gestão e
187     // manipulação do prédio
188 }
189
190 //Classe com a chamada da Main
191 public class ExampleUML {
192
193     public static void main(String[] args) {
194         // Criar um prédio com 2 andares
195         Predio predio = new Predio(2);
196
197         // Adicionar um andar adicional ao prédio
198         predio.adicionarAndar();
199
200         // Adicionar salas aos andares
201         predio.adicionarSalaAoAndar(0); // Adiciona uma sala ao primeiro
202             andar
203         predio.adicionarSalaAoAndar(0); // Adiciona uma sala ao primeiro
204             andar
205         predio.adicionarSalaAoAndar(1); // Adiciona uma sala ao segundo

```

```

197         andar
        predio.adicionarSalaAoAndar(2); // Adiciona uma sala ao terceiro
        andar
198
199         // Adicionar mesas às salas
200         predio.adicionarMesaSala(0, 0, new Mesa(null)); // Adiciona uma
        mesa na primeira sala do primeiro andar
201         predio.adicionarMesaSala(1, 0, new Mesa(null)); // Adiciona uma
        mesa na primeira sala do segundo andar
202         predio.adicionarMesaSala(2, 0, new Mesa(null)); // Adiciona uma
        mesa na primeira sala do terceiro andar
203
204         // Visualizar a configuração do prédio
205         for (int i = 0; i < predio.obterNumeroDeSalasNoAndar(0); i++) {
206             int numeroMesas = predio.getNumeroMesasporSala(0, i);
207             System.out.println("Andar 1 - Sala " + (i + 1) + " tem " +
                numeroMesas + " mesas.");
208         }
209
210         for (int i = 0; i < predio.obterNumeroDeSalasNoAndar(1); i++) {
211             int numeroMesas = predio.getNumeroMesasporSala(1, i);
212             System.out.println("Andar 2 - Sala " + (i + 1) + " tem " +
                numeroMesas + " mesas.");
213         }
214
215         for (int i = 0; i < predio.obterNumeroDeSalasNoAndar(2); i++) {
216             int numeroMesas = predio.getNumeroMesasporSala(2, i);
217             System.out.println("Andar 3 - Sala " + (i + 1) + " tem " +
                numeroMesas + " mesas.");
218         }
219     }
220 }
221 }

```

3 Exercícios Práticos

3.1 Java

Implemente em Java (um de cada vez!) uma série de classes que modelem o seguinte problema: uma editora produz Livros por meio de diversos Capítulos, cada qual escrito por um autor. Para que seja feito um software orientado a objetos para essa editora armazenar e manipular seus livros será preciso implementar as classes Capítulo e Livros. Veja o diagrama abaixo. Implemente as classes em arquivos separados, e faça um programa que utilize a classe Livro incluindo um ou mais livros. As classes deverão ter as seguintes características: a) Capítulo possuirá os atributos: título, número de páginas (número inteiro maior ou igual a 1) e sobrenome do autor. Deverá fornecer um construtor que inicialize todos as variáveis membro, e métodos de acesso ao título do capítulo, autor e número de páginas. Métodos modificadores não são necessários b) Livro possuirá os atributos: editor (sobrenome da pessoa que organizou o livro), título do livro, ano (número inteiro maior ou igual 2011), edição (número inteiro maior que 0), preço e capítulos (um ou mais objetos Capítulo). A classe livro deverá conter um construtor para inicializar as variáveis membro, exceto os capítulos. Deverá fornecer um método para inserir objetos capítulo, e um

método para imprimir as informações do livro (que deverá imprimir editor, título, ano, edição preço e uma lista de capítulos).

1. Crie uma classe **Tempo** com três atributos: horas, minutos e segundos. Crie dois construtores: um para inicializar os atributos com valor 0 e outro para inicializar os atributos com valores passados como argumentos. Crie métodos para:
 - a. Funcionar como getter e setter;
 - b. Imprimir os atributos no formato hh:mm:ss;
 - c. Subtrair dois objetos e colocar o resultado no objeto que o chamou;
 - d. Somar que soma dois objetos e colocar o resultado no objeto que o chamou;
 - e. Sobrecarregue este último método para que retorne um objeto com o resultado da operação.
2. Crie uma classe **Estacionamento** para armazenar dados de um estacionamento. Os atributos devem representar a placa e modelo do carro além da hora de entrada e saída do estacionamento. Utilize dois objetos da classe **Tempo** criada no exercício anterior. Crie métodos para:
 - a. Funcionar como getter e setter;
 - b. Inicializar os dados com vazio ou zero;
 - c. Imprimir os dados de um carro estacionado;
 - d. Calcular e retornar o valor a ser pago pelo carro estacionado. Considere o preço de R\$1,50 por hora. Utilize o método da classe **Tempo**.
3. Implemente em Java (um de cada vez!) uma série de classes que modelem o seguinte problema: uma editora produz Livros por meio de diversos Capítulos, cada qual escrito por um autor. Para que seja feito um software orientado a objetos para essa editora armazenar e manipular seus livros será preciso implementar as classes **Capítulo** e **Livros**. Veja o diagrama abaixo. Implemente as classes em arquivos separados, e faça um programa que utilize a classe **Livro** incluindo um ou mais livros. As classes deverão ter as seguintes características:
 - a) **Capítulo** possuirá os atributos: título, número de páginas (número inteiro $i \geq 1$) e sobrenome do autor. Deverá fornecer um construtor que inicialize todos as variáveis membro, e métodos de acesso ao título do capítulo, autor e número de páginas. Métodos modificadores não são necessários.
 - b) **Livro** possuirá os atributos: editor (sobrenome da pessoa que organizou o livro), título do livro, ano (número inteiro $i \geq 2011$), edição (número inteiro $i \geq 0$), preço e capítulos (um ou mais objetos **Capítulo**). A classe **livro** deverá conter um construtor para inicializar as variáveis membro, exceto os capítulos. Deverá fornecer um método para inserir objetos capítulo, e um método para imprimir as informações do livro (que deverá imprimir editor, título, ano, edição preço e uma lista de capítulos).
4. Crie uma classe **Fração**, com as seguintes características:
 - a. Crie um construtor que evita o valor zero ou negativo como denominador;
 - b. Sobrecarregue os operadores de adição, subtração, multiplicação e divisão desta classe;
5. Faça uma classe **Vetor** com um método soma que receba dois vetores de inteiros e retorne um terceiro vetor com a soma dos valores dos dois vetores passados como argumento. Sobrecarregue esse método para que também possa receber dois vetores de números decimais e retorne um terceiro vetor com a soma dos valores dos dois vetores passados como argumento.

6. Implemente uma classe `Circulo` com um método `calculaArea` que receba o raio do círculo e retorne a área. Sobrecarregue esse método para que também possa receber o diâmetro do círculo e retorne a área.
7. Crie uma classe `Matriz` com um método `soma` que receba duas matrizes de inteiros e retorne uma terceira matriz com a soma dos valores das duas matrizes passadas como argumento. Sobrecarregue esse método para que também possa receber duas matrizes de números decimais e retorne uma terceira matriz com a soma dos valores das duas matrizes passadas como argumento.

3.2 UML

Dos exercícios 1 a 4, na criação dos diagramas, em alguns casos, a decisão de qual a melhor alternativa para modelagem pode ser sua. Fique a vontade.

Crie um diagrama de classes para:

1. Um sistema de gerenciamento de tarefas com as seguintes características:
 - Tarefas com nome, descrição, data de criação e data de conclusão.
 - Atribuição de tarefas a usuários.
 - Usuários com nome e e-mail, capazes de criar, editar e concluir tarefas.
2. Um sistema de locadora de filmes com as seguintes características:
 - Filmes com título, ano de lançamento, diretor e gênero.
 - Aluguel de filmes por clientes por um período de tempo, com pagamento de taxa.
 - Criação de novos filmes e clientes, e consulta do histórico de aluguéis de um cliente.
3. Um sistema de gerenciamento de projetos com as seguintes características:
 - Projetos com nome, descrição, data de início e data de término, compostos por várias tarefas.
 - Tarefas com nome, descrição, data de criação e data de conclusão, atribuídas a membros da equipe.
 - Membros da equipe com nome e e-mail, capazes de criar, editar e concluir tarefas.
 - Criação de novos projetos, membros da equipe e atribuição de tarefas a eles.
4. Um sistema de vendas online com as seguintes características:
 - Clientes realizando pedidos de vários produtos.
 - Produtos com nome, preço e descrição.
 - Criação de novos produtos, realização de pedidos pelos clientes e cálculo do valor total da compra.
 - Consulta do histórico de pedidos de um cliente.

Analise os códigos abaixo e crie os seus diagramas de classes UML. Explique esses diagramas e escolhas.

Código 1

```
1 public class Pessoa {
2     private String nome;
3     private int idade;
4     private Endereco endereco;
5
6     public Pessoa(String nome, int idade, Endereco endereco) {
```

```

7         this.nome = nome;
8         this.idade = idade;
9         this.endereco = endereco;
10    }
11
12    public String getNome() {
13        return nome;
14    }
15
16    public void setNome(String nome) {
17        this.nome = nome;
18    }
19
20    public int getIdade() {
21        return idade;
22    }
23
24    public void setIdade(int idade) {
25        this.idade = idade;
26    }
27
28    public Endereco getEndereco() {
29        return endereco;
30    }
31 }
32
33 public class Endereco {
34     private String rua;
35     private int numero;
36     private String cidade;
37     private String estado;
38
39     public Endereco(String rua, int numero, String cidade, String estado) {
40         this.rua = rua;
41         this.numero = numero;
42         this.cidade = cidade;
43         this.estado = estado;
44     }
45
46     public String getRua() {
47         return rua;
48     }
49
50     public void setRua(String rua) {
51         this.rua = rua;
52     }
53
54     public int getNumero() {
55         return numero;
56     }
57

```

```

58     public void setNumero(int numero) {
59         this.numero = numero;
60     }
61
62     public String getCidade() {
63         return cidade;
64     }
65
66     public void setCidade(String cidade) {
67         this.cidade = cidade;
68     }
69
70     public String getEstado() {
71         return estado;
72     }
73
74     public void setEstado(String estado) {
75         this.estado = estado;
76     }
77 }
78
79 public class Empresa {
80     private String nome;
81     private ArrayList<Funcionario> funcionarios;
82
83     public Empresa(String nome) {
84         this.nome = nome;
85         funcionarios = new ArrayList<Funcionario>();
86     }
87
88     public String getNome() {
89         return nome;
90     }
91
92     public void setNome(String nome) {
93         this.nome = nome;
94     }
95
96     public void adicionarFuncionario(Funcionario funcionario) {
97         funcionarios.add(funcionario);
98     }
99
100    public void removerFuncionario(Funcionario funcionario) {
101        funcionarios.remove(funcionario);
102    }
103
104    public ArrayList<Funcionario> getFuncionarios() {
105        return funcionarios;
106    }
107 }
108

```

```

109 public class Funcionario {
110     private String nome;
111     private int idade;
112     private Empresa empresa;
113
114     public Funcionario(String nome, int idade, Empresa empresa) {
115         this.nome = nome;
116         this.idade = idade;
117         this.empresa = empresa;
118     }
119
120     public String getNome() {
121         return nome;
122     }
123
124     public void setNome(String nome) {
125         this.nome = nome;
126     }
127
128     public int getIdade() {
129         return idade;
130     }
131
132     public void setIdade(int idade) {
133         this.idade = idade;
134     }
135
136     public Empresa getEmpresa() {
137         return empresa;
138     }
139 }

```

Código 2

```

1     public class Escola {
2         private String nome;
3         private ArrayList<Turma> turmas;
4
5         public Escola(String nome) {
6             this.nome = nome;
7             turmas = new ArrayList<Turma>();
8         }
9
10        public String getNome() {
11            return nome;
12        }
13
14        public void setNome(String nome) {
15            this.nome = nome;
16        }
17
18        public void adicionarTurma(Turma turma) {

```

```

19         turmas.add(turma);
20     }
21
22     public void removerTurma(Turma turma) {
23         turmas.remove(turma);
24     }
25
26     public ArrayList<Turma> getTurmas() {
27         return turmas;
28     }
29 }
30
31 public class Turma {
32     private int ano;
33     private String letra;
34     private ArrayList<Aluno> alunos;
35     private Professor professor;
36
37     public Turma(int ano, String letra, Professor professor) {
38         this.ano = ano;
39         this.letra = letra;
40         this.professor = professor;
41         alunos = new ArrayList<Aluno>();
42     }
43
44     public int getAno() {
45         return ano;
46     }
47
48     public void setAno(int ano) {
49         this.ano = ano;
50     }
51
52     public String getLetra() {
53         return letra;
54     }
55
56     public void setLetra(String letra) {
57         this.letra = letra;
58     }
59
60     public void adicionarAluno(Aluno aluno) {
61         alunos.add(aluno);
62     }
63
64     public void removerAluno(Aluno aluno) {
65         alunos.remove(aluno);
66     }
67
68     public ArrayList<Aluno> getAlunos() {
69         return alunos;

```

```

70     }
71
72     public Professor getProfessor() {
73         return professor;
74     }
75 }
76
77 public class Aluno {
78     private String nome;
79     private int idade;
80     private Turma turma;
81
82     public Aluno(String nome, int idade, Turma turma) {
83         this.nome = nome;
84         this.idade = idade;
85         this.turma = turma;
86         turma.adicionarAluno(this);
87     }
88
89     public String getNome() {
90         return nome;
91     }
92
93     public void setNome(String nome) {
94         this.nome = nome;
95     }
96
97     public int getIdade() {
98         return idade;
99     }
100
101     public void setIdade(int idade) {
102         this.idade = idade;
103     }
104
105     public Turma getTurma() {
106         return turma;
107     }
108 }
109
110 public class Professor {
111     private String nome;
112     private ArrayList<Turma> turmas;
113
114     public Professor(String nome) {
115         this.nome = nome;
116         turmas = new ArrayList<Turma>();
117     }
118
119     public String getNome() {
120         return nome;

```

```
121     }
122
123     public void setNome(String nome) {
124         this.nome = nome;
125     }
126
127     public void adicionarTurma(Turma turma) {
128         turmas.add(turma);
129     }
130
131     public void removerTurma(Turma turma) {
132         turmas.remove(turma);
133     }
134
135     public ArrayList<Turma> getTurmas() {
136         return turmas;
137     }
138 }
```